

Clase 116 — Arquitectura x86/x64 y lenguaje ensamblador

Parte: 5 — Explotación de sistemas y binarios · Fuente: Erickson, Hacking: The Art of Exploitation, 2e · Intel SDM 🛠 Duración estimada: 120 min · Nivel: Fundamentos

Objetivo

Entender cómo la CPU x86/x64 ejecuta instrucciones y traducir mentalmente entre C y ensamblador. Al final sabrás qué son los registros de propósito general, cómo se representan las instrucciones en memoria, la diferencia entre las sintaxis Intel y AT&T, y podrás leer el desensamblado de una función sencilla identificando el prólogo, el cuerpo y el epílogo. Es la base sin la cual el resto de la parte (overflows, ROP, reversing) no se sostiene.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Enumerar** los registros de propósito general de x86 (32 bits) y x64 (64 bits) y su propósito.
- 2. **Distinguir** las sintaxis Intel y AT&T y convertir instrucciones simples entre ambas.
- 3. **Compilar** un programa en C a ensamblador y **relacionar** cada línea con su origen.
- 4. **Desensamblar** un binario con `objdump` y localizar el prólogo y epílogo de una función.
- 5. **Explicar** el ciclo fetch-decode-execute y el papel de `RIP / EIP` y `RFLAGS`.

Temas

#	Tema	Por qué importa
1	Modos de operación (real, protegido, largo)	Define el tamaño de registro y direccionamiento
2	Registros GPR: RAX...R15, EAX...EDI	Son el "espacio de trabajo" de todo exploit
3	RIP/EIP y RFLAGS	Controlar RIP = controlar la ejecución
4	Endianness (little-endian)	Cómo se colocan los bytes de una dirección en memoria
5	Sintaxis Intel vs AT&T	Cambia el orden operando y los prefijos
6	Instrucciones básicas: mov, add, lea, cmp, jmp, call	Vocabulario mínimo para leer código
7	Prólogo/epílogo de función	Punto donde se guarda y restaura el marco
8	Del C al ASM (gcc -S)	Puente entre lo que escribes y lo que corre

Definiciones y características

- **Registro de propósito general (GPR):** almacenamiento rapidísimo dentro de la CPU. En x64 hay 16 de 64 bits (`RAX` – `R15`); sus mitades de 32 bits son `EAX`, etc. *Característica clave:* operar sobre `EAX` pone a cero los 32 bits altos de `RAX`.
- **RIP / EIP (instruction pointer):** apunta a la siguiente instrucción a ejecutar. *Clave:* no se escribe directamente con `mov`; se altera con `call`, `ret`, `jmp` — de ahí su valor para el atacante.
- **RFLAGS:** registro de banderas (ZF, SF, CF, OF...) que refleja el resultado de operaciones y guía los saltos condicionales. *Clave:* `cmp` no guarda resultado, solo actualiza banderas.
- **Little-endian:** el byte menos significativo se almacena primero. *Clave:* la dirección `0x08049000` se escribe en memoria como `00 90 04 08`.
- **Opcode / operandos:** cada instrucción es uno o más bytes de código de operación seguidos de sus operandos. *Clave:* las instrucciones x86 tienen longitud variable (1 a 15 bytes).
- **Sintaxis AT&T vs Intel:** AT&T usa `mov $0x1, %eax` (origen→destino, prefijos `%` / `$`); Intel usa `mov eax, 1` (destino←origen). *Clave:* GDB por defecto usa AT&T; se puede cambiar a Intel.

Herramientas y preparación

Trabaja en una **VM Linux aislada** (por ejemplo Ubuntu/Kali x86-64) que usarás durante toda la parte.

```
sudo apt update
sudo apt install -y build-essential gdb gcc-multilib nasm binutils
# Verifica versiones
gcc --version && objdump --version && nasm --version
```

Para ver desensamblado con sintaxis Intel de forma cómoda añade a `~/.gdbinit`: `set disassembly-flavor intel`.

Laboratorio guiado

Entorno propio: todo se compila y ejecuta en tu VM.

1. Crea `suma.c`:

```
c int suma(int a, int b) { return a + b; } int main(void) { return suma(3, 4); }
```

2. Genera ensamblador legible con sintaxis Intel:

```
bash gcc -O0 -S -masm=intel suma.c -o suma.s cat suma.s
```

Localiza `push rbp` / `mov rbp, rsp` (prólogo) y `pop rbp` / `ret` (epílogo).

3. Compila y desensambla el binario:

```
bash gcc -O0 suma.c -o suma objdump -d -M intel suma | sed -n '/<suma>:/,/ret/p'
```

4. Observa cómo se pasan los argumentos: en x64 el primero va en `EDI` y el segundo en `ESI` (System V ABI). Anota qué instrucción hace la suma (`add`).

5. Escribe tu primer ASM puro con NASM (`hola.asm`) que solo termine con `exit(42)`:

```
asm section .text global _start _start: mov rax, 60 ; syscall exit mov rdi, 42 ; código de salida syscall
```

```
bash nasm -f elf64 hola.asm -o hola.o && ld hola.o -o hola ./hola; echo $? # imprime 42
```

6. Compara la salida de `objdump -d hola` con lo que escribiste para confirmar la traducción a opcodes.

Ejercicios

1. Convierte a sintaxis AT&T: `mov eax, 5, add rbx, rax, lea rax, [rbp-0x4]`.
2. Escribe en little-endian los bytes de la dirección `0x00401136`.
3. Modifica `hola.asm` para que devuelva la suma de dos inmediatos usando `add`.
4. Compila `suma.c` con `-O2` y explica por qué el desensamblado es más corto.
5. Identifica en un `objdump` cualquiera tres instrucciones de salto y di qué bandera consultan.
6. Escribe una función C con un `if` y localiza el `cmp` + `jne` correspondiente en el ASM.

Reto verificable

Escribe en NASM un programa que calcule $(7 * 6) - 5$ usando solo registros e instrucciones aritméticas y devuelva el resultado como código de salida.

Criterio de aceptación: `./programa; echo $?` imprime **37**, y `objdump -d` muestra al menos una instrucción `imul` / `mul` y una `sub`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>objdump</code> muestra AT&T y no lo entiendes	Añade <code>-M intel</code> , o <code>set disassembly-flavor intel</code> en GDB
<code>ld: cannot find entry symbol _start</code>	Falta <code>global _start</code> ; NASM no exporta el símbolo por defecto
Segfault al salir del NASM	Usaste <code>ret</code> sin stack válido; termina con la syscall <code>exit</code>
Registros de 32 bits "no cambian" los altos	Escribir <code>EAX</code> sí pone a cero el alto de <code>RAX</code> ; revisa qué mitad usas
El ASM de <code>-O2</code> no coincide con tu C	El optimizador reordena/elimina; compila con <code>-O0</code> para aprender

Preguntas frecuentes

 **¿Necesito aprender x86 de 32 bits si todo es de 64?** Sí: muchos retos de CTF y binarios legacy son de 32 bits, y las convenciones (paso de argumentos por stack) son distintas y didácticas.

 **¿Intel o AT&T?** Aprende a leer ambas. Intel suele ser más clara para principiantes; AT&T es el default de muchas herramientas GNU.

? **¿Tengo que memorizar todas las instrucciones?** No. Domina un núcleo de ~20 instrucciones y consulta el *Intel SDM* para el resto.

Referencias

- Erickson, J. *Hacking: The Art of Exploitation*, 2e, cap. 0x2. No Starch Press.
- Intel® 64 and IA-32 Architectures Software Developer's Manual — <https://www.intel.com/sdm>
- (X86 assembly) OSDev Wiki — <https://wiki.osdev.org/X86-64>
- Compiler Explorer (godbolt) para ver C↔ASM — <https://godbolt.org/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiendo clase

Clase 117 - El stack, los registros y las convenciones de llamada